

NeoMutt, Neovim, and abook: a practical configuration guide

Contents

1. Recommendation and current setup	2
2. How to add mail plugins with lazy.nvim	3
A separate place for mail-only settings and mappings	4
3. Historical addresses with telescope-notmuch.nvim	4
What it adds	4
Installation	5
Dependency checks	5
4. Saved aliases with vim-mutt-aliases	6
When it is useful	6
Installation and keys	6
5. abook completion inside Neovim	6
Why this is worth adding	6
Correction concerning mutt-query-complete.vim	6
A shell-free alternative requiring no extra plugin	7
6. Structured editing with mail-headers.nvim	9
What you gain and what you must provide	9
Step A: register the mail parser using your existing Tree-sitter setup	9
Step B: install the header plugin, initially disabled	9
Step C: create an address file	10
Step D: add custom buffer mappings	10
7. Your abook settings explained	11
One actual correction: sort_field	11
Every current setting	11
Proposed cleaned configuration	12
Daily contact workflow	13
8. Groups in abook	13
What the article contributes	13
Example metadata	13
Proposed wrapper	14

9. What to take from the two blog posts	16
10. NeoMutt repository improvements	16
Accurate Notmuch tags	16
Sync and notifications	17
HTML and attachment handling	17
Mail composition	17
Repository maintenance	18
11. Installation order and verification	18
Suggested order	18
Test without sending mail	18
Troubleshooting	19
Reverting an optional feature	19
12. Source directory	20
Plugins and Neovim	20
Mail tools and articles	20
Local configuration reviewed	20
Checks performed when preparing this guide	20

Prepared for Soham's configuration, reviewed on 30 September 2026.

This guide explains the changes to make; writing this document has not installed plugins or changed the live configuration. Examples use your current paths and lazy.nvim layout. Installation snippets are proposed configuration, not a claim that the complete integration has been tested in your running editor. Use the verification steps before relying on them for real correspondence.

1. Recommendation and current setup

Keep the offline-mail architecture you already have: mbsync downloads and synchronizes Maildir, NeoMutt reads and composes, msmtplib sends, Notmuch indexes, abook stores maintained contacts, and Neovim edits drafts.

Your Neovim configuration is `/home/soham/.config/nvim`, not `~/.config/neovim`.

Component	Observed configuration	Consequence
Plugin manager	lazy.nvim, with explicit <code>require("plugins....")</code> entries in <code>init.lua</code>	A new plugin file must be registered; simply creating it is insufficient
Telescope	Configured with plenary and native fuzzy sorting	An address picker fits your existing interface
Completion	nvim-cmp with LSP, snippets, buffer, and paths	No dedicated email contact source exists yet
Tree-sitter	New API: <code>require("nvim-treesitter").setup()</code>	Use the modern mail-parser registration, not the legacy <code>configs.setup</code> recipe
Draft editor	<code>nvim -f</code> , with <code>edit_headers</code> enabled	Header completion can work directly in drafts

Component	Observed configuration	Consequence
Address prompts	Tab for aliases; Ctrl-T for abook	Already useful before entering Neovim
Save contact	<code>ga</code> in index/pager	Saves to abook, not the alias file
Aliases	Three definitions at review time	Alias completion has less immediate coverage than historical lookup
Notmuch	Default config points into this repository	A Neovim Notmuch subprocess can find the same database
Contacts on disk	<code>~/.abook</code> links to the repository's abook directory	Edit the repository configuration; do not create a second address book

Recommended responsibilities:

- **abook:** people whose contact details you intentionally maintain.
- **Notmuch:** finding an address from previous correspondence.
- **NeoMutt aliases:** short names and a few stable recipient lists.
- **abook groups:** recurring membership lists, when needed.
- **mail-headers:** editing the recipients already present in a draft.

Do not install all plugins merely because they are compatible in principle. Start with historical lookup and/or abook completion; add structured header editing when you have a use for it.

2. How to add mail plugins with lazy.nvim

Your entry point is `init.lua`. Create this proposed file:

`/home/soham/.config/nvim/lua/plugins/mail.lua`

Initially give it the following contents:

```
return {
  -- Add the plugin specification tables from sections 3, 4, and 6 here.
}
```

Then add one entry inside the existing `require("lazy").setup({ ... })` list:

```
require("plugins.mail"),
```

Do not replace the rest of `init.lua`, and do not add another independent call to `lazy.setup`. The snippets below are table entries to insert inside the returned list. Their trailing commas separate entries.

Use `:Lazy` to inspect installation. Use `:Lazy install` to install missing enabled plugins, then restart Neovim. Avoid an indiscriminate update of all existing plugins just to add one mail tool. Preserve your lockfile and record tested plugin revisions after the integration works.

For Codeberg repositories, use `url = "https://...git"`. For GitHub repositories, the "owner/repository" shorthand is sufficient. `ft = "mail"` defers loading until a mail buffer. `init` is where globals needed before a plugin loads belong. Sources: lazy.nvim specification, lazy loading.

A separate place for mail-only settings and mappings

Create this proposed file:

```
/home/soham/.config/nvim/after/ftplugin/mail.lua
```

Unlike your explicitly listed plugin specifications, `after/ftplugin/mail.lua` is found automatically by Neovim’s filetype-plugin mechanism. It runs for each mail buffer, after the normal mail ftplugin.

Start it with:

```
vim.opt_local.spell = true
vim.opt_local.spelllang = "en_us"
vim.opt_local.textwidth = 72
vim.opt_local.formatoptions:append({ "t", "c", "q" })
-- Ordinary mail paragraphs do not need programming-language indentation.
vim.opt_local.indentexpr = ""
vim.opt_local.smartindent = false

local function map(mode, lhs, rhs, description)
  vim.keymap.set(mode, lhs, rhs, {
    buffer = true,
    silent = true,
    desc = description,
  })
end

-- Append only the mappings for features you actually enabled.
```

These wrapping settings assume `unset text_flow`d in NeoMutt. Do not apply them as a complete flowed-text configuration.

The `buffer = true` option is important: mail shortcuts should not take over your coding buffers. `desc` makes mappings easier to inspect with your existing which-key integration and `:map` commands. Change the left-hand key string to customize a shortcut. In mappings below, `i` means Insert mode and `n` means Normal mode.

Your Ctrl-Space, Tab, Ctrl-N/P, Ctrl-Y, and Ctrl-L already have completion or snippet roles. Leave those alone initially. Source: Neovim Lua keymaps.

3. Historical addresses with telescope-notmuch.nvim

What it adds

This is the easiest of the three requested plugins to add to your setup. It exposes addresses from Notmuch through Telescope and inserts a selected result. It does not read messages or perform structured header editing. The inspected implementation queries asynchronously and inserts one selected entry. Its default query collects senders; recipients appearing only in To/Cc/Bcc require a different query. Sources: README, implementation.

Use it for “I remember receiving a message from this person, but I never saved their address.” In the

review, its underlying query found 381 addresses. That is a snapshot, not a permanent expected count.

Installation

Insert this table in `lua/plugins/mail.lua`:

```
{
  url = "https://codeberg.org/JoshuaCrewe/telescope-notmuch.nvim.git",
  ft = "mail",
  dependencies = { "nvim-telescope/telescope.nvim" },
  config = function()
    require("telescope").load_extension("notmuch")
  end,
},
```

Append to `after/ftplugin/mail.lua`:

```
map("i", "<C-x><C-p>", function()
  require("telescope").extensions.notmuch.notmuch(
    require("telescope.themes").get_cursor({})
  )
end, "Mail: find historical address")
```

Alternatively, use the documented command mapping:

```
-- Alternative to the mapping above; choose one.
map("i", "<C-x><C-p>",
  "<Cmd>Telescope notmuch theme=cursor<CR>",
  "Mail: find historical address")
```

Put the cursor at the desired insertion position in a recipient header, invoke the picker, search, and select. The picker text is a search within the picker; this is not automatic replacement of a partially typed word in the draft. Manage commas in the draft yourself.

Dependency checks

```
command -v notmuch jq
notmuch count '*'
notmuch address --format=json --deduplicate=address '*' | jq 'length'
```

These are read-only. Your machine already has the executables and a working default Notmuch configuration. No separate Neovim environment override was needed during review.

For a future custom picker, the following includes both senders and recipients:

```
notmuch address --format=json --deduplicate=address \
  --output=sender --output=recipients '*'
```

Do not assume this can be enabled through a documented plugin option: the inspected plugin hardcodes its query. A custom extension or maintained fork would be needed. Recipient extraction may take longer. Source: `notmuch-address`.

4. Saved aliases with vim-mutt-aliases

When it is useful

Install this if you regularly use short aliases and want them in the editor. It reads your alias file, not `abook`. Saving someone with `ga` therefore does not make that contact available here. It uses native `completefunc` completion, separate from `nvim-cmp`. Sources: README, plugin setup.

Installation and keys

Add to `lua/plugins/mail.lua`:

```
{
  "Konfekt/vim-mutt-aliases",
  ft = "mail",
  init = function()
    vim.g.muttaliases_file = vim.fn.expand(
      "~/.config/neomutt/modules/aliases.muttrc"
    )
    vim.g.muttaliases_filetypes = { "mail" }
  end,
},
```

Its native Insert-mode trigger is `Ctrl-X`, `Ctrl-U`. No mapping is required. If you want a custom alias, append:

```
map("i", "<C-x><C-a>", "<C-x><C-u>", "Mail: complete saved alias")
map("n", "<leader>ma", "<Cmd>EditAliases<CR>", "Mail: edit aliases")
```

For example, given alias `colleague Example Person <person@example.org>`, type the alias in `To:`, then invoke completion. All example addresses in this guide are fictional.

Inspect setup with:

```
:setlocal completefunc?
:verbose imap <C-x><C-a>
```

Expect the completion function to reference `muttaliases`. If not, restart Neovim and reopen a mail buffer after installation. To disable the plugin, set `enabled = false` on its spec and remove its custom mappings.

5. abook completion inside Neovim

Why this is worth adding

Your existing NeoMutt query command is `abook --mutt-query %s`. Using `abook` in Neovim means a contact saved through `ga` becomes available while editing headers too. This is a closer match to your maintained contacts than installing an aliases-only plugin.

Correction concerning `mutt-query-complete.vim`

Earlier recommendations mentioned `mutt-query-complete.vim`. Its inspected autoload implementation substitutes the completion text directly into a shell command and calls `system()` on that

string. This can mishandle spaces, quotes, and shell metacharacters. Automatic discovery also assumes a particular format for NeoMutt's query output.

I do **not** recommend deploying that implementation unchanged. Explicitly specifying its command does not fix the interpolation issue. Use a corrected fork or the small argument-list implementation below. Do not install that plugin together with this implementation: both would own `omnifunc`.

A shell-free alternative requiring no extra plugin

Create this proposed module:

```
/home/soham/.config/nvim/lua/mail_abook.lua
```

```
local M = {}

-- This list can later be replaced by the group-aware Python command
-- from section 8. Each argument remains a separate process argument.
M.command = { "abook", "--mutt-query" }

function M.complete(findstart, base)
  if findstart == 1 then
    local before = vim.fn.getline("."):sub(1, vim.fn.col(".") - 1)
    if not before:match("^([Tt] [Oo]:")
      and not before:match("^([Cc] [Cc]:")
      and not before:match("^([Bb] [Cc] [Cc]:") then
      return -3
    end
    -- Complete after the last colon or comma; exclude leading whitespace.
    local start = before:match(".*[:,]()")
    if not start then return -3 end
    while before:sub(start, start):match("%s") do
      start = start + 1
    end
    return start - 1 -- Vim expects a zero-based byte index.
  end

  local args = vim.deepcopy(M.command)
  table.insert(args, base or "")
  local ok, result = pcall(function()
    return vim.system(args, { text = true }):wait(3000)
  end)
  if not ok then
    vim.notify("Could not start the mail contact query", vim.log.levels.WARN)
    return {}
  end
  if result.code ~= 0 then
    vim.notify("Contact query failed or found no matches", vim.log.levels.INFO)
    return {}
  end
end
```

```

local matches, seen = {}, {}
-- Query protocol: first line is status, later lines are tab-separated.
local lines = vim.split(result.stdout or "", "\n", { plain = true })
for i = 2, #lines do
    local address, name = lines[i]:match("^([^\t]+)\t([^\t]*)")
    if address and not address:find("[%c<>]") then
        address = vim.trim(address)
        name = vim.trim(name or "")
        if address ~= "" and not seen[address:lower()] then
            seen[address:lower()] = true
            local escaped = name:gsub("\\", "\\"):gsub("'", "\\'")
            local word = name == "" and address
                or ('"' .. escaped .. "' <" .. address .. ">")
            table.insert(matches, {
                word = word,
                abbr = name == "" and address or name,
                menu = address,
            })
        end
    end
end
return matches
end

return M

```

Append to after/ftplugin/mail.lua:

```

_G.MailAbookComplete = function(findstart, base)
    return require("mail_abook").complete(findstart, base)
end
vim.bo.omnifunc = "v:lua.MailAbookComplete"
map("i", "<C-x><C-o>", "<C-x><C-o>", "Mail: complete abook contact")

```

The mapping is nonrecursive by default, so the right side invokes native omni completion. Type part of a name after To:, then Ctrl-X, Ctrl-O. This implementation supports ordinary single-line To/Cc/Bcc header editing. It deliberately does not attempt to parse quoted commas, folded headers, or group expressions containing commas; use a full query prompt for those.

It blocks for at most the query timeout while retrieving a small address book. It is not an asynchronous nvim-cmp source. This is a modest bridge, not a complete mail-address parser. Your alias plugin can coexist because it uses `completetfunc`, while this uses `omnifunc`.

Keep native completion separate initially. If you later want nvim-cmp integration, introduce an omni source deliberately and test its trigger behavior; an existing omnifunc does not automatically become an nvim-cmp source. Sources: Neovim process API, completion documentation.

6. Structured editing with mail-headers.nvim

What you gain and what you must provide

Use this when rearranging recipients is frequent. It offers header navigation, address deletion, and Telescope-based additions/moves. It requires a compatible mail Tree-sitter parser. Its contact picker reads a text file containing one address per line, so it needs an export rather than directly querying abook. Sources: README, implementation.

The inspected source does not create missing destination headers. Have **To:**, **Cc:**, and **Bcc:** present before moving recipients. A move deletes from the old header before adding to the new one; a failure needs immediate undo and inspection. Test with a scratch message first.

Step A: register the mail parser using your existing Tree-sitter setup

In `/home/soham/.config/nvim/lua/plugins/ide/treesitter.lua`, inside its existing config function, register this before the existing `ts.install({...})` call:

```
local function register_mail_parser()
    require("nvim-treesitter.parsers").mail = {
        install_info = {
            url = "https://codeberg.org/ficd/tree-sitter-mail",
            branch = "master",
            queries = "queries/mail",
        },
    }
end

vim.api.nvim_create_autocmd("User", {
    group = vim.api.nvim_create_augroup("MailParserRegistration", { clear = true }),
    pattern = "TSUpdate",
    callback = register_mail_parser,
})

register_mail_parser()
```

Add "mail" to that existing installation list. Restart, then run `:TSInstall mail` if it has not installed automatically. Wait for completion and reopen the mail buffer.

Keep your existing Tree-sitter setup; do not copy the header plugin README's old `branch = "master"` dependency example over it. Tree-sitter's modern API and its frozen legacy API are different. Your machine has a compiler and tree-sitter executable, but verify version compatibility before updating the installed Tree-sitter plugin. Sources: mail grammar's Neovim instructions, custom parser registration.

Step B: install the header plugin, initially disabled

Add to `lua/plugins/mail.lua`:

```
{
    url = "https://codeberg.org/pmassot/mail-headers.nvim.git",
    enabled = false, -- Change to true after preparing the parser and address file.
```

```

ft = "mail",
dependencies = {
  "nvim-telescope/telescope.nvim",
  "nvim-treesitter/nvim-treesitter",
},
init = function()
  vim.g.mailheaders_settings = {
    addresses = vim.fn.expand("~/cache/neomutt/addresses.txt"),
    set_mappings = false,
    picker_mappings = {
      moveTo = "<C-t>",
      moveCc = "<C-c>",
      moveBcc = "<C-b>",
      delete = "<C-d>",
    },
  }
end,
},

```

Do not add `opts = {}` or `config = true` here: this plugin's documented configuration uses a global table, not a conventional `setup()` call. The picker action keys operate inside its header-editing picker. They are separate from draft-buffer mappings and can be changed independently.

Step C: create an address file

For an initial historical-address export, run this in Bash:

```

set -euo pipefail
umask 077
mkdir -p "$HOME/.cache/neomutt"
mail_addresses_tmp=$(mktemp "$HOME/.cache/neomutt/addresses.XXXXXX")
trap 'rm -f -- "$mail_addresses_tmp"' EXIT
notmuch address --format=json --deduplicate=address '*' \
  | jq -r '.[ ] | .["name-addr"]' \
  | sort -u > "$mail_addresses_tmp"
mv -- "$mail_addresses_tmp" "$HOME/.cache/neomutt/addresses.txt"

```

This exports senders, matching the simple historical picker. It does not export abook groups. Refresh after successful mail indexing if you adopt it. Keep generated contact lists outside Git. A future combined export should prefer curated abook names and deduplicate by email address, not merely by whole display lines.

Step D: add custom buffer mappings

Only append these after enabling the plugin:

```

local function header_action(name, argument)
  return function()
    require("mail-headers")[name](argument)
  end
end

```

end

```
map("n", "<leader>ms", header_action("goto_header", "Subject"), "Mail: edit subject")
map("n", "<leader>mat", header_action("mailbox_picker", "To"), "Mail: add To")
map("n", "<leader>mac", header_action("mailbox_picker", "Cc"), "Mail: add Cc")
map("n", "<leader>mab", header_action("mailbox_picker", "Bcc"), "Mail: add Bcc")
map("n", "<leader>mt", header_action("move_current_mailbox", "To"), "Mail: move to To")
map("n", "<leader>mc", header_action("move_current_mailbox", "Cc"), "Mail: move to Cc")
map("n", "<leader>mb", header_action("move_current_mailbox", "Bcc"), "Mail: move to Bcc")
map("n", "<leader>md", header_action("delete_current_mailbox"), "Mail: delete recipient")
map("n", "<leader>met", header_action("header_edit_picker", "To"), "Mail: edit To list")
map("n", "<leader>mec", header_action("header_edit_picker", "Cc"), "Mail: edit Cc list")
map("n", "<leader>meb", header_action("header_edit_picker", "Bcc"), "Mail: edit Bcc list")
```

<leader>ma from the optional alias mapping is a prefix of these add-recipient mappings. If you install both, change the alias-editor mapping to <leader>mA to avoid waiting for a longer key sequence.

In an add-recipient picker, use Tab to select multiple entries and Enter to add them. In an edit-existing-header picker, use its configured move/delete action keys instead. Your general Telescope Ctrl-L selection mapping may not mean “perform a header move”; use the explicit action keys.

7. Your abook settings explained

The authoritative reference used here is your installed `man 5 abookrc`, with `man 1 abook` for commands. The project’s home is abook; a web-readable manual copy is `abookrc(5)`. Installed documentation takes precedence when versions differ.

Your configuration lives at `/home/soham/.config/neomutt/abook/abookrc`. The `~/.abook` symlink already connects this to abook.

One actual correction: `sort_field`

Your current `set sort_field=true` is not a boolean setting. It should name a field, such as:

```
set sort_field=name
```

This selects the field for abook’s “sort by field” action. It is not a general automatic-sort-on-startup switch. This correction was found when checking the installed manual in detail.

Every current setting

Setting	Meaning	Recommendation
<code>autosave=true</code>	Saves the address book when abook exits	Keep one occurrence; yours appears twice. Do not assume every keystroke is immediately saved to disk
<code>sort_field=true</code>	Incorrect value for a field-name option	Change to <code>name</code> , or <code>nick</code> if you prefer nickname ordering

Setting	Meaning	Recommendation
<code>view CONTACT = name, email</code>	Fields shown in the CONTACT tab	Keep; optionally include <code>nick</code>
<code>view ADDRESS = email</code>	Another tab containing email	Redundant with CONTACT; replace with postal fields or omit
<code>view OTHER = url</code>	OTHER tab contains contact website	Add <code>notes</code> and optional group membership
<code>index_format="{name:22} {email:70}"</code>	Format of list columns, with field width limits	Fine on a wide terminal; narrow terminals may benefit from a smaller email column
<code>preserve_fields=keep</code>	Keeps fields that the current configuration does not recognize or display	Keep, especially while introducing custom fields
<code>show_all_emails=true</code>	Show all addresses belonging to a contact in the list	Keep if seeing work/personal alternatives is useful
<code>mutt_command=neomutt</code>	Mutt client launched by abook's compose action	Keep
<code>mutt_return_all_emails=true</code>	Returns all matching contact email addresses to mail queries	Keep for explicit selection; it does not mean "send to all" automatically
<code>print_command=lp</code>	Command for printing the address book	Only relevant if you print contacts; it is not NeoMutt's message-printing setting
<code>www_command=lynx</code>	Opens a contact's website in Lynx	Change to <code>xdg-open</code> if you want your desktop browser
<code>use_ascii_only=false</code>	Uses non-ASCII interface characters	Keep with your Unicode terminal
<code>add_email_prevent_duplicates=true</code>	Avoids duplicating an address already present	Keep; this is not a complete person-merging/deduplication system
<code>show_cursor=false</code>	Hides the terminal cursor in the main list	Pure display preference; keep

The display setting `show_all_emails` and the query setting `mutt_return_all_emails` are independent. You can show all addresses while returning fewer in queries. Choose deliberately before adding group expansion so a person's work and personal accounts are not both selected unintentionally.

Proposed cleaned configuration

```
# Data handling
set autosave=true
set preserve_fields=all
set add_email_prevent_duplicates=true

# Sorting and display
set sort_field=name
set index_format="{name:26} | {email:55}"
set show_all_emails=true
set use_ascii_only=false
set show_cursor=false
```

```
# External commands
set mutt_command=neomutt
set mutt_return_all_emails=true
set www_command=xdg-open
set print_command=lpr

# Custom field: metadata only until a query wrapper uses it
field email_lists = "Email groups", string

view CONTACT = name, email, nick
view ADDRESS = address, address2, city, state, zip, country
view PHONE = phone, workphone, mobile
view OTHER = url, notes, email_lists
```

Adding a custom field does not itself implement group queries. `preserve_fields=all` preserves hidden data but does not make it visible; a `view` controls visibility.

Daily contact workflow

1. In NeoMutt, use your existing `ga` to add the sender.
2. Open `abook` to correct the name, add a nickname, or assign groups.
3. Exit `abook` so autosave writes the result.
4. In a NeoMutt recipient prompt, type a fragment and `Ctrl-T`.
5. In a Neovim draft, use the `abook` omni-completion bridge if installed.

Your `a` key creates a NeoMutt alias; it does not add to `abook`. Keep this distinction clear. Your contact backup script encrypts both data sources, and your Git hooks back them up when committing. That is not continuous backup between commits.

8. Groups in `abook`

What the article contributes

The `abook` groups article proposes a custom membership field and a wrapper that recognizes `g/group-name`, while ordinary queries still go to `abook`. It is useful for repeated, overlapping recipient sets. It does not create a server mailing list or a new email address.

The article's implementation needs changes: exact rather than substring matching, duplicate suppression, record-aware parsing, a policy for multiple addresses, and the initial status line required by NeoMutt. Source for the protocol: external address queries.

Example metadata

```
[0]
name=Example Person
email=person@example.org
email_lists=seminar,reading-group
```

```
[1]
```

```
name=Another Person
email=another@example.org
email_lists=seminar
```

Use lowercase group labels and avoid commas inside a label. The following proposed implementation treats labels case-insensitively, ignores surrounding whitespace, and uses the first email address of each contact for groups. Arrange that address intentionally in abook. Ordinary searches still obey abook's own query configuration.

Proposed wrapper

Save as `/home/soham/.config/neomutt/scripts/contact-query.py` if you choose to implement groups:

```
#!/usr/bin/env python3
"""NeoMutt contact query: ordinary abook search or g/group,other-group."""
import configparser
import os
from pathlib import Path
import subprocess
import sys

def main():
    if len(sys.argv) != 2:
        print("Expected one contact query")
        return 2
    query = sys.argv[1]
    if not query.startswith("g/"):
        return subprocess.run(["abook", "--mutt-query", query]).returncode

    groups = {part.strip().casefold() for part in query[2:].split(",")}
    groups.discard("")
    if not groups:
        print("Specify at least one group after g/")
        return 1

    # Override is useful for isolated tests; the normal path follows your symlink.
    path = Path(os.environ.get("ABOOK_GROUPS_FILE", "~/abook/addressbook")).expanduser()
    database = configparser.ConfigParser(interpolation=None, strict=True)
    try:
        with path.open(encoding="utf-8") as stream:
            database.read_file(stream)
    except (OSError, UnicodeError, configparser.Error) as error:
        print("Could not read the abook group database")
        print(str(error), file=sys.stderr)
        return 1

    result = {}
```

```

for section in database.sections():
    if not section.isdecimal():
        continue
    entry = database[section]
    memberships = {
        item.strip().casefold()
        for item in entry.get("email_lists", "").split(",")
    }
    if not groups.intersection(memberships):
        continue
    addresses = [a.strip() for a in entry.get("email", "").split(",") if a.strip()]
    if not addresses:
        continue
    address = addresses[0]
    if any(c in address for c in "\r\n\t<>") or "@" not in address:
        continue
    name = " ".join(entry.get("name", "").split()) or address
    result.setdefault(address.casefold(), (address, name))

rows = sorted(result.values(), key=lambda row: (row[1].casefold(), row[0].casefold()))
print(f"{len(rows)} addresses found:")
for address, name in rows:
    print(f"{address}\t{name}")
return 0 if rows else 1

if __name__ == "__main__":
    raise SystemExit(main())

```

This is intended for abook's simple INI-style data, not arbitrary email-header syntax. It neither sends mail nor rewrites contacts. The embedded version passed isolated fixtures for exact matching, overlapping-group deduplication, field order, case handling, and empty results. It still needs a real query-menu check before deployment. Group results are a union, not an intersection: a contact in either selected group is returned once.

Change NeoMutt's query command to:

```
set query_command = "python3 ~/.config/neomutt/scripts/contact-query.py %s"
```

Keep %s unquoted in the NeoMutt command template: NeoMutt handles query substitution quoting. Do not transfer the unsafe interpolation behavior of a third-party Vim plugin to this interface.

For the custom Neovim bridge, replace `M.command` with:

```

M.command = {
    "python3",
    vim.fn.expand("~/config/neomutt/scripts/contact-query.py"),
}

```

The Python wrapper does not need executable permission when invoked through `python3`. Use `Q` in NeoMutt for `g/seminar,reading-group`, then choose/tag the returned recipients. Ctrl-T prompt

completion and the simple Neovim bridge both use address separators; a multi-group expression containing commas is better entered into the full query prompt.

A group query returns candidates, not permission to automatically send mail. Review To/Cc/Bcc on the compose screen. No group setup is necessary if your correspondence is predominantly one-to-one.

9. What to take from the two blog posts

Mutt and Friends discusses urlview, HTML conversion, abook, printing, and indexed search. You already have equivalents for most of that toolchain. Keep urlview, Lynx, abook, and Notmuch. There is no reason to introduce mairix alongside your working Notmuch integration merely to follow the article. Add message printing only if you actually need it.

Your installed urlview documents **WRAP yes**, but treat wrapping as something to test with a troublesome link, not a guarantee that broken URLs in arbitrary messages are repaired. Your current URL opening command is already **xdg-open**.

The article's uppercase **A** contact shortcut would conflict with your existing mark-all-read macro. Keep **ga**. Your explicit `<pipe-message>abook --add-email-quiet` binding already supplies the message correctly.

The groups article adds a distinct capability: maintained membership lists. Adopt it only if recurring groups justify the extra configuration. It does not replace historical lookup or recipient editing.

10. NeoMutt repository improvements

Accurate Notmuch tags

At review time, 2,462 messages had **inbox** without an **INBOX** copy, 1,962 Trash messages lacked **trash**, and 16 Junk messages lacked **spam**. These are dated observations, not expected future counts.

`[new] tags=unread;inbox;` in your Notmuch configuration initializes tags irrespective of folder. Introduce folder-aware reconciliation after **notmuch new**, remove stale folder-derived tags after moves, preserve personal tags, and define a policy for messages with copies in multiple folders. Apply that reconciliation once to the existing index as well. Maildir flag synchronization does not supply all folder semantics. Source: Notmuch configuration.

Read-only checks:

```
notmuch count 'tag:inbox and not folder:INBOX'
notmuch count 'folder:Trash and not tag:trash'
notmuch count 'folder:Junk and not tag:spam'
```

Do not blindly run broad retagging commands until you decide whether messages with both Inbox and Trash copies should be excluded. That choice changes search visibility.

Sync and notifications

Your timer was active when checked. The current script continues after a failed mbsync and announces the unread backlog again on each run. Add shared locking, explicit failure handling, folder-tag reconciliation, and a notification policy based on newly arrived messages. If mbsync partially succeeds, decide deliberately whether to index the downloaded subset while still reporting the sync failure.

After that, make **S** start the systemd service rather than run the whole script in the foreground. Useful diagnostics are:

```
systemctl --user status mailsync.timer mailsync.service
journalctl --user -u mailsync.service -n 50
```

Do not run a full synchronization merely to test a configuration snippet: your mbsync configuration includes **Expunge Both**, so synchronization can propagate deletions.

HTML and attachment handling

Your mailcap now contains Zathura PDF/DjVu/PostScript entries, image **xdg-open**, Office viewers, archive previews, and media handlers. Keep the specific DjVu image MIME entries before **image/***.

Remove the old duplicate Lynx rule at the very top of mailcap so your browser-first HTML pair is effective. Add to neomuttrc:

```
auto_view text/html
alternative_order text/plain text/html
```

Optionally add to the binding module:

```
bind attach p view-pager
```

p in the attachment menu previews converted text. Your **bind compose p postpone-message** and **bind index p recall-message** remain intact because bindings are menu-specific.

x-neomutt-keep prevents premature deletion for detached applications but retains temporary files. Complete this with a background attachment helper and age-based cleanup policy. Do not assume **xdg-open**'s exit means the image window has closed. Source: NeoMutt MIME support.

Mail composition

Keep **set edit_headers**. Initially keep **autoedit** off so you can use **abook** in NeoMutt's normal prompts. After editor completion works, enabling **autoedit** is an optional preference: it skips those initial prompts and takes you directly into the draft.

For conventional 72-column text, use **unset text_flow**. A flowed-text setup needs intentional editor support; enabling the NeoMutt option alone does not add the required soft-wrap markers. Source: `format=flowed` guidance.

Your current **sendmail_wait=0** means wait for **msmtp** to finish, not immediate background delivery. Keep it if you want errors reported as part of sending. It is unrelated to GUI attachment backgrounding.

Repository maintenance

- Change the decorations typo `encryptped` to `encrypted` if you want that tag transformed.
- The temp directory currently uses `neomut` rather than `neomutt`. It exists, so this is consistency cleanup, not a demonstrated failure.
- Align the SMTP template with the working log path and `set_from_header` setting.
- Make generators resolve templates relative to their script location and write atomically. They currently depend on the working directory and can replace your live symlink targets.
- Update README to document the actual Maildir/msmtp setup, symlinks, service setup, and contact restoration.
- Keep encrypted contact backups and the existing hooks. Generated plaintext address exports belong in cache, not in Git.
- Do not modify working SMTP host/TLS settings merely because they differ from a generic tutorial.

11. Installation order and verification

Suggested order

1. Fix abook `sort_field`, keep one autosave setting, and choose contact views.
2. Repair Notmuch folder tags and sync reliability as a separate configuration task.
3. Register `plugins.mail` and create the mail-specific ftplugin.
4. Add Telescope historical lookup and/or the shell-free abook bridge.
5. Add aliases only if you use them regularly.
6. Add groups only after choosing multi-address and membership behavior.
7. Add mail-headers after the parser and address export work.
8. Complete attachment launching and documentation cleanup.

Test without sending mail

Create `/tmp/neomutt-guide-test.eml` with fictional data:

```
From: Me <me@example.org>
To: Alice <alice@example.org>, Bob <bob@example.org>
Cc: Carol <carol@example.org>
Bcc:
Subject: Local configuration test
```

This is a local test draft. Do not send it.

Open it in Neovim and check:

```
:setfiletype mail
:setlocal filetype? spell? textwidth? completfunc? omnifunc?
:Lazy
:verbose imap <C-x><C-p>
:verbose imap <C-x><C-o>
```

For header editing, also run:

```
:lua print(pcall(vim.treesitter.get_parser, 0, "mail"))
:InspectTree
```

Verify each behavior individually:

- Historical picker inserts the intended address at the intended location.
- abook completion handles a partial name and a name containing spaces.
- Alias completion expands an existing alias through Ctrl-X, Ctrl-U.
- Header moves retain every intended recipient exactly once and preserve separators.
- Add-recipient picker multi-selection behaves as expected.
- Mail-only keys do not appear in an unrelated Lua file.
- A normal NeoMutt draft still postpones with **p** in the compose menu.

If header editing fails, undo immediately and inspect the entire header block. Do not use a failed move as a starting point for subsequent moves. Keep the optional plugin disabled until the parser behavior is confirmed.

Troubleshooting

Symptom	Check
Plugin file exists but plugin is absent	Did you add <code>require("plugins.mail")</code> to the existing lazy setup?
Nothing loads in a scratch buffer <code>EditAliases</code> is missing	Check <code>:set ft?</code> ; the specs are mail-only Install/enable the alias plugin, then restart and open a mail buffer
No historical addresses	Test <code>notmuch count '*'</code> in the same environment as Neovim
abook sees different contacts	Inspect the <code>~/.abook</code> symlink and any explicit <code>--datafile</code> override
Custom completion works but Ctrl-Space does not show contacts	Native omni completion and nvim-cmp are separate
Mail parser missing	Check parser registration, compiler/CLI availability, and <code>:TSInstall mail</code> output
Header picker empty	Check that the address export exists and has one address per line
Move to Cc fails	Ensure <code>Cc:</code> exists and the parser recognizes the message
HTML still opens as Lynx text with <code>m</code>	Remove the duplicate first HTML rule
Repeated new-mail notifications	Fix tag policy and notification state, not the Neovim plugins

Reverting an optional feature

Remove its buffer mappings, disable its plugin spec, and restart Neovim. Remove the custom `omnifunc` assignment if disabling the abook bridge. Keep contact data and the alias file; plugin removal should not delete them. Preserve your old `abookrc` before replacing its layout. Keep your lazy lockfile so a known-working plugin revision can be restored.

12. Source directory

Plugins and Neovim

- telescope-notmuch.nvim
- telescope-notmuch implementation
- vim-mutt-aliases
- mail-headers.nvim
- mail-headers implementation
- mail Tree-sitter grammar
- nvim-treesitter modern branch
- mutt-query-complete implementation reviewed for limitations
- lazy.nvim specification
- Neovim Lua API
- Neovim insert completion

Mail tools and articles

- abook project
- abookrc manual, web copy
- Local primary references: `man 5 abookrc`, `man 1 abook`, `man 1 notmuch-address`, `man 1 urlview`.
- NeoMutt external queries
- NeoMutt reference
- NeoMutt MIME handling
- Notmuch configuration
- Notmuch address queries
- Mutt and Friends
- Using Mutt and Abook with Email Groups

Local configuration reviewed

- NeoMutt main configuration
- NeoMutt bindings
- abook configuration
- Notmuch configuration
- Sync script
- Mailcap
- Neovim initialization
- Neovim completion
- Neovim Tree-sitter configuration

Checks performed when preparing this guide

- All Lua code blocks were syntax-checked with the installed Neovim in clean headless mode; plugin table fragments were wrapped as specifications for that check.
- The proposed abook configuration was accepted by the installed abook using a fictional address book.

- The group wrapper passed isolated fixture tests without reading or modifying your real contacts.
- The Neovim abook bridge was exercised against that fictional address book, including completion boundaries and literal handling of shell-like query text.
- The Telescope extension export name was checked against upstream source.
- Plugin installation, parser compilation, interactive Telescope behavior, and mail-headers edits were not run. Those remain the manual verification steps above.

Only this Markdown guide was added to the repository. Test fixtures were created under `/tmp`; no live configuration, contacts, or mail were changed.